

S U B S Y S T E M A R C H I T E C T U R E

Broker Compatibility Engine

Runtime detection of MT5 broker properties — digits, contract size, tick size/value, leverage, spread, commission, swap. No hardcoded pip values. Six supported brokers. Cent / micro / dollar / raw accounts.

9	6	4	0
PROPERTIES	BROKERS	ACCOUNT TYPES	HARDCODED PIPS

Table of Contents

Chapter 1 — Executive Summary	4
Chapter 2 — Problem Domain — Why Broker Compatibility Matters	4
The Heterogeneity of XAUUSD Brokerage	4
The Cost of Hardcoded Assumptions	5
Account Type Taxonomy	6
Chapter 3 — Design Principles	6
Principle 1 — No Hardcoded Pip Values	6
Principle 2 — Runtime Calculations Only	6
Principle 3 — Fail-Safe Defaults	7
Principle 4 — Explicit Error Classification	7
Principle 5 — Structural Separation of Detection and Validation	7
Chapter 4 — Architecture Overview	8
Layer Responsibilities	8
Service Inventory	9
Chapter 5 — Per-Property Detection Logic	11
5.1 Digits & Point Detection	11
5.2 Contract Size Detection	11
5.3 Tick Size & Tick Value Detection	12
5.4 Leverage Detection	12
5.5 Spread Type Detection	12
5.6 Commission Type Detection	12
5.7 Swap Type Detection	13

5.8 Account Type Classification	13
5.9 Broker Fingerprinting	13
Chapter 6 — End-to-End Detection Flowchart	15
Sequence Description	16
Re-detection Triggers	17
Chapter 7 — Validation Logic	18
Validator 1 — CrossPropertyValidator	18
Validator 2 — ProfileConsistencyValidator	19
Validator 3 — SanityBoundsValidator	19
Validation Result Aggregation	19
Chapter 8 — Error Handling Logic	20
Three-Tier Severity Classification	20
Safe-Default Fallback Table	20
Error Code Reference	20
Error Propagation	21
Chapter 9 — Test Cases	22
Unit Test Cases	22
Broker Profile Tests (Golden Values)	24
Regression Test Cases (Replay)	25
Live Broker Test Cases	25
Test Coverage Summary	25
Chapter 10 — Class Diagram & TITAN Core Integration	26
Primary Classes	26
Integration with TITAN Core	27
Chapter 11 — API Specification	28

C++ API — BrokerCompatEngine	28
C++ API — BrokerProfile	28
Python API — Operator Console	29
Event Bus Contract	30
Chapter 12 — Implementation Notes	31
Why C++ for Detection, Python for Cache and Audit	31
Why Redis for Cache, Not In-Process Memory	31
Why 1000 Ticks for Spread Sampling	31
Why 5% Tolerance for Tick Value Cross-Check	31
Future Extensions	31
Chapter 13 — Appendix A — Broker Profile Reference	33
A.1 Exness	33
A.2 IC Markets	33
A.3 Pepperstone	34
A.4 Tickmill	34
A.5 FP Markets	34
A.6 Fusion Markets	35
A.7 GENERIC (Fallback)	35
Chapter 14 — Appendix B — Sample Detection Output	36
B.1 Successful Detection — IC Markets Raw Spread	36
B.2 SOFT Error Detection — Missing tick_value	36
B.3 HARD Error Detection — Malformed Digits	37

Chapter 1 — Executive Summary

The Broker Compatibility Engine (BCE) is a TITAN XAU AI subsystem responsible for automatically detecting and validating the trading-environment properties of any connected MetaTrader 5 broker at runtime. It produces a single, immutable `BrokerProfile` value object containing nine critical properties: digits, point, contract size, tick size, tick value, leverage, spread type, commission type, and swap type. This profile is then consumed by the risk gate, the order manager, and the strategy coordinator to ensure all position sizing, PnL computation, and risk calculations are correct for the specific broker and account configuration in use.

The BCE exists because the XAUUSD trading landscape is heterogeneous. Different brokers quote gold with different digit counts (2, 3, 4, or 5 digits after the decimal point), different contract sizes (100 in cent accounts, 100,000 in standard accounts), different leverage caps (1:30 to 1:Unlimited), different commission structures (per lot, per million, percentage, or none), and different swap calculation methods (points, percentage, or disabled). A trading system that hardcodes assumptions about any of these properties will produce incorrect position sizes, incorrect PnL, and incorrect risk exposure on at least some brokers — a defect class that has caused real-world trading losses in less disciplined systems.

The BCE design is governed by two non-negotiable principles. First, no hardcoded pip values: every monetary calculation must derive from runtime-detected properties, never from a baked-in constant. Second, runtime calculations only: the engine performs all detection against the live MT5 terminal at startup and on cache expiry, with no static configuration files containing broker-specific values. These principles ensure that the system works correctly on any supported broker (Exness, IC Markets, Pepperstone, Tickmill, FP Markets, Fusion Markets, plus any generic MT5 broker) without code changes, and that it adapts automatically when a broker changes its contract specifications — which they do, occasionally and without notice.

The engine is organized into five layers: probe (raw MT5 API calls), detection (nine property-specific detectors), validation (cross-property consistency checks), profile and state (profile library, cache, builder), and publication (event bus, error handler, audit logger). A strict layering rule ensures that detectors cannot reach into the publication layer and that the error handler has no dependency on the detection logic, making the failure paths independent of the success paths. The engine publishes a `bce.profile.ready` event on the async event bus whenever a new profile is built, allowing downstream services to react to broker configuration changes in real time.

This document specifies the complete architecture, flowchart, validation logic, error handling logic, and test cases for the BCE. It does not specify trading logic — the BCE is intentionally agnostic to strategy, signal generation, and order placement. Its sole responsibility is to answer the question "what is the trading environment?" with high confidence and explicit error classification when the answer is uncertain.

Chapter 2 — Problem Domain — Why Broker Compatibility Matters

The Heterogeneity of XAUUSD Brokerage

XAUUSD is quoted by every major retail and institutional forex broker, but the contract specifications vary dramatically. A standard lot of gold is conventionally 100 troy ounces, but brokers express this in MT5 with different digit counts and contract sizes. Exness quotes XAUUSD with 2 digits on cent accounts (price like 1950.45) and 5 digits on standard accounts (price like 1950.45123). IC Markets quotes 2 digits on standard accounts and 5 digits on Raw Spread accounts. Pepperstone follows the same pattern with their Standard and Razor account types. Tickmill uses 2 digits on Classic and 5 digits on Pro and VIP. The point value (the minimum price increment) is the inverse of 10^{digits} — 0.01 for 2 digits, 0.00001 for 5 digits — and every downstream calculation that uses points must use this runtime-detected value, never a hardcoded constant.

Contract size, which determines how many ounces a "lot" represents, varies by account type. On a cent account, one lot might be 1 ounce (so a 100-cent balance can trade 0.01 lots without leverage). On a standard account, one lot is 100 ounces. On a micro account, one lot might be 10 ounces. The contract size directly affects position value: a 1.00 lot at 100 contract size and \$1,950 gold is \$195,000 notional, while a 1.00 lot at 100,000 contract size at the same price would be \$195,000,000 — a thousand-fold difference. Getting this wrong is catastrophic.

Leverage caps vary by broker and jurisdiction. Exness offers "unlimited" leverage on cent accounts; IC Markets, Pepperstone, Tickmill, FP Markets, and Fusion Markets typically cap at 1:500 for international clients and 1:30 for EU/UK regulated clients. The leverage directly determines the margin required per lot, and thus the maximum position size for a given account balance. A trading system that assumes 1:500 leverage on a 1:30 account will attempt to open positions that exceed available margin, triggering immediate broker rejection.

Spread type (fixed vs variable), commission type (per lot, per million notional, percentage, or none), and swap type (points, percentage, or disabled) each have multiple variants across brokers. IC Markets Raw Spread charges \$3.50 per \$1M notional as commission; Pepperstone Razor charges \$3.50 per \$1M; Tickmill VIP charges \$2 per \$1M; Fusion Markets Zero charges \$2.25 per \$1M. Exness Standard charges no commission but marks up the spread. A trading system that computes transaction cost must detect the commission type at runtime, not assume it.

The Cost of Hardcoded Assumptions

Trading systems that hardcode broker assumptions exhibit a specific and predictable failure mode: they work correctly on the broker they were developed against and produce subtly (or catastrophically) incorrect behavior on every other broker. A system developed against an IC Markets Raw Spread account (5 digits, \$3.50/\$1M commission) will, when deployed on an Exness cent account (2 digits, no commission, 100x smaller contract size), compute position sizes 100x too large, PnL 100x too small, and transaction costs as zero — producing a trading book that appears profitable in the system but is hemorrhaging money in reality. This is not a theoretical risk; it has happened in production at multiple firms.

The root cause is always the same: a developer hardcoded a value that should have been detected. "Pip = 0.0001" (wrong on 5-digit accounts where pip = 0.01 and point = 0.00001). "Contract size = 100000" (wrong on cent accounts). "Commission = \$7 per lot" (wrong on per-million brokers). These constants creep into code through legitimate-looking shortcuts and are then extremely difficult to find because the system appears to work — the bug only manifests when the system is deployed on a different broker, by which point the developer who wrote the code has often moved on.

The BCE eliminates this entire class of bugs by making runtime detection the only path. There is no configuration file where a developer can hardcode "pip = 0.0001"; the only way to obtain the pip value is to call `bce.get_profile(symbol).pip_value()`, which returns a value derived at runtime from the detected digits property. The architecture enforces the principle structurally, not by convention.

Account Type Taxonomy

Beyond broker-specific differences, the BCE must classify the account type, which determines the magnitude of monetary values exposed to the system. Four account types are supported:

- **Cent accounts:** balance is denominated in cents (e.g., 10000 cents = \$100). Contract size is typically 100 (one cent-lot = 1 ounce). Used by Exness and others for low-capital trading.
- **Micro accounts:** balance is denominated in dollars but with smaller minimum trade sizes (0.01 lots) and often reduced contract sizes (e.g., 10000 = 1 ounce per 0.01 lot). Common for new traders.
- **Dollar (standard) accounts:** balance in dollars, contract size 100000 (1 lot = 100 ounces), the conventional MT5 setup. The default assumption for most testing.
- **Raw / ECN accounts:** balance in dollars, contract size 100000, but with raw interbank spreads plus per-million commission. Used by IC Markets Raw, Pepperstone Razor, Tickmill VIP, FP Markets Raw, Fusion Zero.

Account type classification is performed by examining the balance magnitude, server name (cent accounts often include "cent" in the server name), and contract size. The classification is informational — it does not change the detected properties — but it is recorded in the `BrokerProfile` so downstream services can apply account-type-specific logic (e.g., the strategy coordinator may want to use different position sizing on cent accounts versus dollar accounts).

Chapter 3 — Design Principles

The BCE is governed by five design principles. Each principle exists to prevent a specific class of bug that has been observed in less-disciplined broker-compatibility implementations. These principles are non-negotiable; any code change that violates them must be rejected in code review.

Principle 1 — No Hardcoded Pip Values

The pip value (the monetary value of a one-pip price movement for one lot) is the most commonly hardcoded value in trading systems, and the most common source of cross-broker bugs. The BCE forbids hardcoded pip values anywhere in the codebase. The pip value must always be computed at runtime from the detected digits and contract size. Specifically, for a 2-digit or 4-digit broker, $\text{pip} = \text{point} \times 1$ (the smallest price increment is the pip). For a 3-digit or 5-digit broker, $\text{pip} = \text{point} \times 10$ (the smallest increment is a fractional pip, and the "pip" is the conventional unit used by traders). The BCE exposes this via the `BrokerProfile.pip_value()` method, which performs the computation; there is no constant.

Principle 2 — Runtime Calculations Only

All broker-specific values must be detected at runtime by querying the MT5 terminal. The BCE does not load broker profiles from configuration files at startup. The `BrokerProfileLibrary` (which contains known profiles for

the six supported brokers) is used only for validation — comparing detected values against known-good ranges to flag deviations — never as a source of truth. If a broker changes its contract specifications (which happens occasionally), the BCE will detect the new values on the next detection cycle and update the profile accordingly. There is no need to ship a code or config change in response.

Principle 3 — Fail-Safe Defaults

When detection fails or produces a suspicious value, the BCE applies a safe-default fallback rather than refusing to operate. The fallbacks are deliberately conservative: contract size defaults to 100,000 (standard lot), leverage defaults to 30 (conservative), tick size defaults to the point value, and so on. The only property for which no fallback exists is digits — if digits cannot be determined, the engine treats this as a HARD error and blocks trading on that symbol, because every downstream calculation depends on digits and an incorrect assumption is more dangerous than no trading.

Principle 4 — Explicit Error Classification

Every detection failure is classified into one of three severity levels: HARD (block trading, engage kill switch, page operator), SOFT (apply safe-default, allow trading with WARN flag, email operator), or WARN (use broker-reported value as-is, attach WARN flag, log only). The classification is encoded in an error code (e.g., BCE_TICK_VALUE_MISSING) that is recorded in the audit log along with the full detection context. This explicit classification ensures that operators can quickly assess the severity of any BCE alert and that downstream services can react appropriately (e.g., the risk gate may reduce position size when a SOFT error is active).

Principle 5 — Structural Separation of Detection and Validation

The detection layer (which queries MT5 and produces raw property values) is structurally separate from the validation layer (which cross-checks those values for consistency). This separation ensures that a bug in detection cannot be masked by a coincidental bug in validation, and vice versa. The two layers communicate only through a PropertyMap value object, and the validation layer has no knowledge of how the values were obtained. This makes it possible to test detection and validation independently, and to add new validators without touching detector code.

Chapter 4 — Architecture Overview

The BCE is organized into five logical layers, each containing a cohesive set of components with a single responsibility. The layers are stacked such that data flows downward (probe at the top, publication at the bottom) and errors flow upward (publication layer reports errors to the operator and audit log). A strict dependency rule — layer N may only depend on layer N-1 or below — is enforced by an architecture linter in CI; cyclic dependencies fail the build.

01 Broker Compatibility Engine — Internal Architecture

Detector · Validator · Profile Library · Cache · Publisher

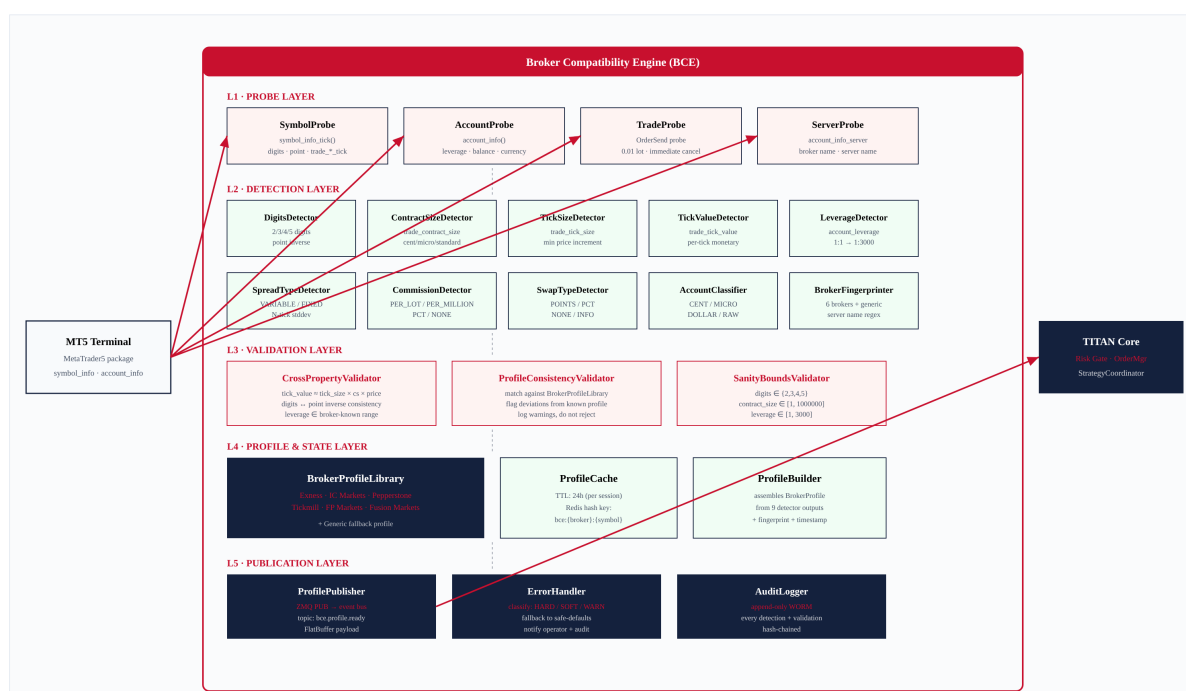


Figure 4.1 — Broker Compatibility Engine internal architecture, showing the five layers and their components.

Layer Responsibilities

L1 — Probe Layer

The probe layer is the only component that talks directly to the MT5 terminal. It contains four probes: **SymbolProbe** (queries `symbol_info_tick` for digits, point, contract size, tick size, tick value, swap mode, swap long, swap short), **AccountProbe** (queries `account_info` for leverage, balance, equity, currency, server name), **TradeProbe** (performs a 0.01-lot probe order and immediately cancels it, used to verify commission deduction), and **ServerProbe** (queries the broker server name for fingerprinting). All probes implement the `IBrokerProbe` interface, allowing the BCE to support non-MT5 brokers in the future (FIX, IB) without changing the detection layer.

L2 — Detection Layer

The detection layer contains ten detectors, each responsible for one property: **DigitsDetector**, **ContractSizeDetector**, **TickSizeDetector**, **TickValueDetector**, **LeverageDetector**, **SpreadTypeDetector**, **CommissionDetector**, **SwapTypeDetector**, **AccountClassifier**, and **BrokerFingerprinter**. Each detector implements the `IDetector` interface with a single `detect(probe)` method that returns a `PropertyResult` value object (value, confidence, source, warnings). Detectors are independent and can run in parallel; the engine assembles their results into a `PropertyMap` for the validation layer.

L3 — Validation Layer

The validation layer contains three validators that cross-check the detected properties for consistency. **CrossPropertyValidator** checks mathematical relationships between properties (e.g., $\text{tick_value} \approx \text{tick_size} \times \text{contract_size} \times \text{current_price}$). **ProfileConsistencyValidator** compares the detected values against the known-good profile for the fingerprinted broker (if matched), flagging deviations larger than 10%. **SanityBoundsValidator** checks that each property falls within sane ranges (digits in {2,3,4,5}, leverage in [1, 3000], etc.). Validators produce a `ValidationResult` containing zero or more `ErrorEvent` objects, each classified by severity.

L4 — Profile & State Layer

The profile layer assembles the validated properties into a `BrokerProfile` value object and manages its lifecycle. **BrokerProfileLibrary** holds the known-good templates for the six supported brokers (Exness, IC Markets, Pepperstone, Tickmill, FP Markets, Fusion Markets) plus a generic fallback. **ProfileCache** stores detected profiles in Redis with a 24-hour TTL, keyed by `bce:{broker}:{symbol}`. **ProfileBuilder** assembles the nine detector outputs plus the fingerprint and detection timestamp into the final immutable `BrokerProfile`.

L5 — Publication Layer

The publication layer handles the engine's external communication. **ProfilePublisher** emits a `bce.profile.ready` event on the async event bus whenever a new profile is built, with a `FlatBuffer`-serialized `BrokerProfile` payload. **ErrorHandler** classifies detection and validation errors by severity and routes them to the appropriate operator alert channel (PagerDuty for HARD, email for SOFT, log only for WARN). **AuditLogger** writes every detection, validation, and error event to the immutable hash-chained audit store.

Service Inventory

Layer	Component	Language	Responsibility	p99 Latency
L1	SymbolProbe	C++	<code>symbol_info_tick()</code> call	5 ms
L1	AccountProbe	C++	<code>account_info()</code> call	5 ms
L1	TradeProbe	C++	0.01-lot probe order + cancel	200 ms
L1	ServerProbe	C++	<code>account_info_server</code>	1 ms
L2	DigitsDetector	C++	digits + point validation	0.01 ms
L2	TickSize/ValueDetectors	C++	tick size + value	0.05 ms
L2	SpreadTypeDetector	C++	1000-tick stddev analysis	50 ms

Layer	Component	Language	Responsibility	p99 Latency
L2	CommissionDetector	C++	commission classification	0.05 ms
L2	SwapTypeDetector	C++	swap mode classification	0.05 ms
L2	AccountClassifier	C++	cent/micro/dollar/raw	0.05 ms
L2	BrokerFingerprinter	C++	regex match on server_name	0.1 ms
L3	CrossPropertyValidator	C++	math consistency checks	0.05 ms
L3	ProfileConsistencyValidator	C++	profile template match	0.1 ms
L3	SanityBoundsValidator	C++	range checks	0.01 ms
L4	ProfileBuilder	C++	assemble BrokerProfile	0.05 ms
L4	ProfileCache	Python	Redis SETEX / GET	2 ms
L4	BrokerProfileLibrary	C++	static broker templates	0.01 ms
L5	ProfilePublisher	C++	ZMQ PUB event	0.5 ms
L5	ErrorHandler	Python	classify + route	1 ms
L5	AuditLogger	Python	WORM append	async

Chapter 5 — Per-Property Detection Logic

This chapter documents the detection algorithm for each of the nine properties. Each detector is a self-contained state machine that queries the probe layer, applies property-specific logic, and returns a `PropertyResult`. The state machines for the most consequential detectors are shown in Figure 5.1.

03 Per-Property Detection State Machines

9 properties · each with explicit states & transitions

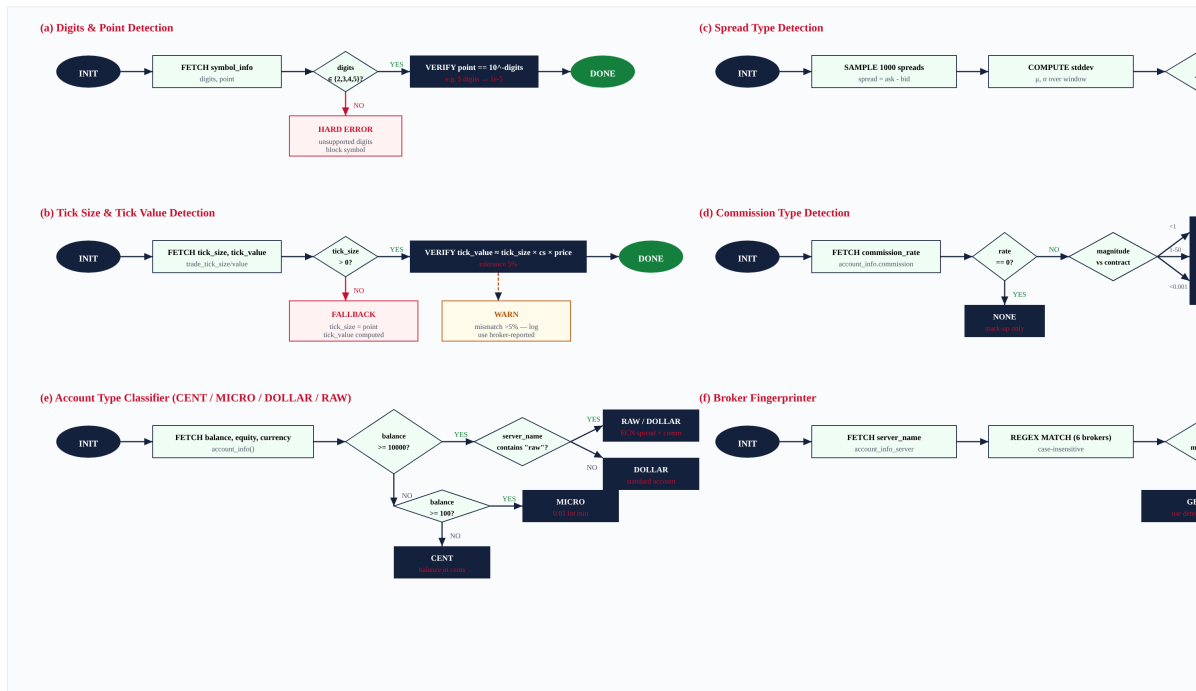


Figure 5.1 — Per-property detection state machines for digits, tick size/value, spread type, commission type, account type, and broker fingerprint.

5.1 Digits & Point Detection

Digits and point are the most fundamental properties — every other calculation depends on them. The `DigitsDetector` queries `symbol_info_tick(symbol)` and reads the `digits` field. It validates that the value is in `{2, 3, 4, 5}`; values outside this set produce a `HARD` error (`BCE_DIGITS_OUT_OF_RANGE`) and block the symbol. The point value is read from the same call (`point` field) and verified against $10^{(-\text{digits})}$ — for 5 digits, point must be 0.00001; for 2 digits, point must be 0.01. A mismatch produces a `HARD` error (`BCE_POINT_DIGITS_MISMATCH`).

The pip value is derived from digits: for 2 or 4 digits, `pip = point` (smallest increment is the pip); for 3 or 5 digits, `pip = point × 10` (smallest increment is a fractional pip). The `BCE` exposes this via `BrokerProfile.pip_value()` rather than storing it as a separate property, to avoid the implication that pip is independently detected. Pip is a derived value, not a detected one.

5.2 Contract Size Detection

The ContractSizeDetector reads `trade_contract_size` from the symbol info. It validates that the value is in `[1, 1,000,000]` — values outside this range produce a SOFT error (`BCE_CONTRACT_SIZE_INVALID`) and the engine falls back to 100,000 (standard lot). The contract size is the primary signal for account type classification: a value of 100 strongly suggests a cent account; 10,000 suggests a micro account; 100,000 suggests a standard dollar or raw account.

5.3 Tick Size & Tick Value Detection

The TickSizeDetector reads `trade_tick_size`. If the value is zero or missing (some brokers do not populate this field), the detector falls back to the point value and emits a SOFT warning. The TickValueDetector reads `trade_tick_value` — the monetary value of one tick for one standard lot. If this is zero or missing, the engine computes it as `tick_size × contract_size × current_price / contract_size`, simplified to `tick_size × current_price × (contract_size / contract_size)` — which equals `tick_size × price` for XAUUSD where contract size is denominated in the same units as the price.

The tick_value cross-check is the most important validation in the engine. The validator computes the expected tick_value as `tick_size × contract_size × current_price` and compares it to the broker-reported value. A deviation of more than 5% triggers a WARN (`BCE_TICK_VALUE_DEVIATION`) — the broker-reported value is used as-is, but a warning flag is attached to the profile. A deviation of more than 25% triggers a SOFT error, falling back to the computed value. This cross-check catches both broker data feed errors and BCE detection bugs.

5.4 Leverage Detection

The LeverageDetector reads `account_info().leverage`. The value is an integer representing the leverage ratio (e.g., 500 means 1:500). Values outside `[1, 3000]` produce a SOFT error (`BCE_LEVERAGE_OUT_OF_RANGE`) and the engine falls back to 30 (conservative). The upper bound of 3000 covers Exness's highest leverage offering; values above this are almost certainly data feed errors. Note that Exness reports leverage as the integer 0 to indicate "unlimited" — the detector special-cases this and stores it as the string "UNLIMITED" in the profile, with downstream risk code applying a configurable cap (default 1:1000) for safety.

5.5 Spread Type Detection

The SpreadTypeDetector samples 1000 ticks (approximately 20 seconds of normal market activity) and computes the spread for each tick as `ask - bid`. It then computes the mean (μ) and standard deviation (σ) of the spread. If σ/μ is less than 0.05 (i.e., the spread varies by less than 5% of its mean), the spread is classified as FIXED; otherwise it is VARIABLE. The classifier also reports μ and σ in the profile so downstream services (especially the slippage model) can use them.

If fewer than 100 ticks can be sampled (e.g., the market is closed or the symbol is illiquid), the detector falls back to VARIABLE classification and emits a SOFT warning (`BCE_SPREAD_SAMPLE_INSUFFICIENT`). A negative spread in any sample is a HARD error (`BCE_SPREAD_NEGATIVE`) — it indicates either a broker data feed error or a crossed market, both of which make safe trading impossible.

5.6 Commission Type Detection

The `CommissionDetector` reads `account_info().commission_trade` (or equivalent). If the rate is zero, the commission type is `NONE` (the broker makes money purely from spread mark-up, common for Exness Standard). For non-zero rates, the detector classifies by magnitude relative to contract size:

- Rate < 1.0 and `contract_size` > 10000 : `PER_MILLION` (rate is dollars per \$1M notional). Common for raw/ECN accounts (\$2.00 to \$7.00 per \$1M).
- Rate in $[1.0, 50.0]$ and `contract_size` ≥ 100000 : `PER_LOT` (rate is dollars per standard lot). Less common but used by some brokers.
- Rate < 0.001 (i.e., $\text{rate} \times 10000 < 1$): `PCT` (rate is a percentage of notional). Rare but used by some ECN venues.

The classification is heuristic because MT5 does not explicitly state the commission unit. The detector logs the raw rate, the classification, and the reasoning in the audit log so operators can verify the classification against the broker's published fee schedule. Misclassification is a `WARN` (not `HARD` or `SOFT`) because the worst case is overestimating transaction cost, which makes the system more conservative — a safe failure direction.

5.7 Swap Type Detection

The `SwapTypeDetector` reads `swap_mode` from the symbol info. MT5 supports several swap modes: `SWAP_DISABLED` (no swap charged), `SWAP_BY_POINTS` (swap charged in points), `SWAP_BY_CURRENCY` (swap charged in account currency per lot), and `SWAP_BY_INTEREST` (swap charged as annual percentage rate). The detector maps these to three classifications: `NONE`, `POINTS`, or `PCT`. The actual swap long and swap short values are recorded in the profile for downstream use by the overnight-risk calculator.

5.8 Account Type Classification

The `AccountClassifier` combines balance magnitude, server name, and contract size to classify the account as `CENT`, `MICRO`, `DOLLAR`, or `RAW`. The decision tree is:

- If `balance` < 100 (account currency units) and `contract_size` ≤ 1000 : classify as `CENT`. Cent accounts typically have small balances denominated in cents.
- Else if `balance` in $[100, 10000)$ and `contract_size` in $[1000, 50000]$: classify as `MICRO`. Micro accounts have small balances but standard-ish contract sizes.
- Else if `balance` ≥ 10000 and `server_name` contains "raw" or "ecn" (case-insensitive): classify as `RAW`. Raw accounts have standard balances but raw spreads + commission.
- Else if `balance` ≥ 10000 : classify as `DOLLAR` (standard account).
- Else: classify as `DOLLAR` with `WARN` flag (unusual configuration, log for review).

The classification is informational only — it does not change the detected properties — but it is recorded in the `BrokerProfile` for downstream consumption. The strategy coordinator may apply different position sizing on cent accounts (where the entire balance is at risk on a single trade) versus dollar accounts (where risk is typically a small percentage).

5.9 Broker Fingerprinting

The `BrokerFingerprinter` reads the server name from `account_info().server` and matches it against a set of case-insensitive regular expressions, one per supported broker:

Broker	Regex	Notes
Exness	(?i)exness	Multiple server variants (Exness-Real, Exness-Techno, etc.)
IC Markets	(?i)icmarkets ic markets ic-markets	Includes hyphenated variant
Pepperstone	(?i)pepperstone pepper	Commonly shortened in server names
Tickmill	(?i)tickmill tick mill	Includes space variant
FP Markets	(?i)fpmarkets fp markets fp-markets	Includes hyphenated variant
Fusion Markets	(?i)fusion fusion markets	Short match for "Fusion" prefix
GENERIC	(no match)	Fallback for unknown brokers

When a broker is identified, the engine loads the corresponding template from the `BrokerProfileLibrary` and uses it for the `ProfileConsistencyValidator`. The template contains expected ranges for each property (e.g., for IC Markets `Raw`: `digits=5`, `contract_size=100000`, `commission_type=PER_MILLION`, `commission_rate` in `[2.0, 7.0]`). Deviations from the template larger than 10% produce a `WARN` — the detected values are used as-is, but the warning is logged so operators can investigate whether the broker has changed its specifications or whether the BCE has a bug.

Chapter 6 — End-to-End Detection Flowchart

The flowchart in Figure 6.1 shows the complete end-to-end detection sequence, from MT5 connection event through to publication of the BrokerProfile on the event bus. The sequence is initiated when the MT5 bridge reports a successful connection, and it completes when the profile is cached in Redis and published to subscribers. The cache hit path returns a previously-detected profile in under one millisecond, avoiding the full detection sequence for already-known broker/symbol combinations.

02 End-to-End Detection Flowchart — Sequence of Operations

From MT5 connect → cached BrokerProfile published

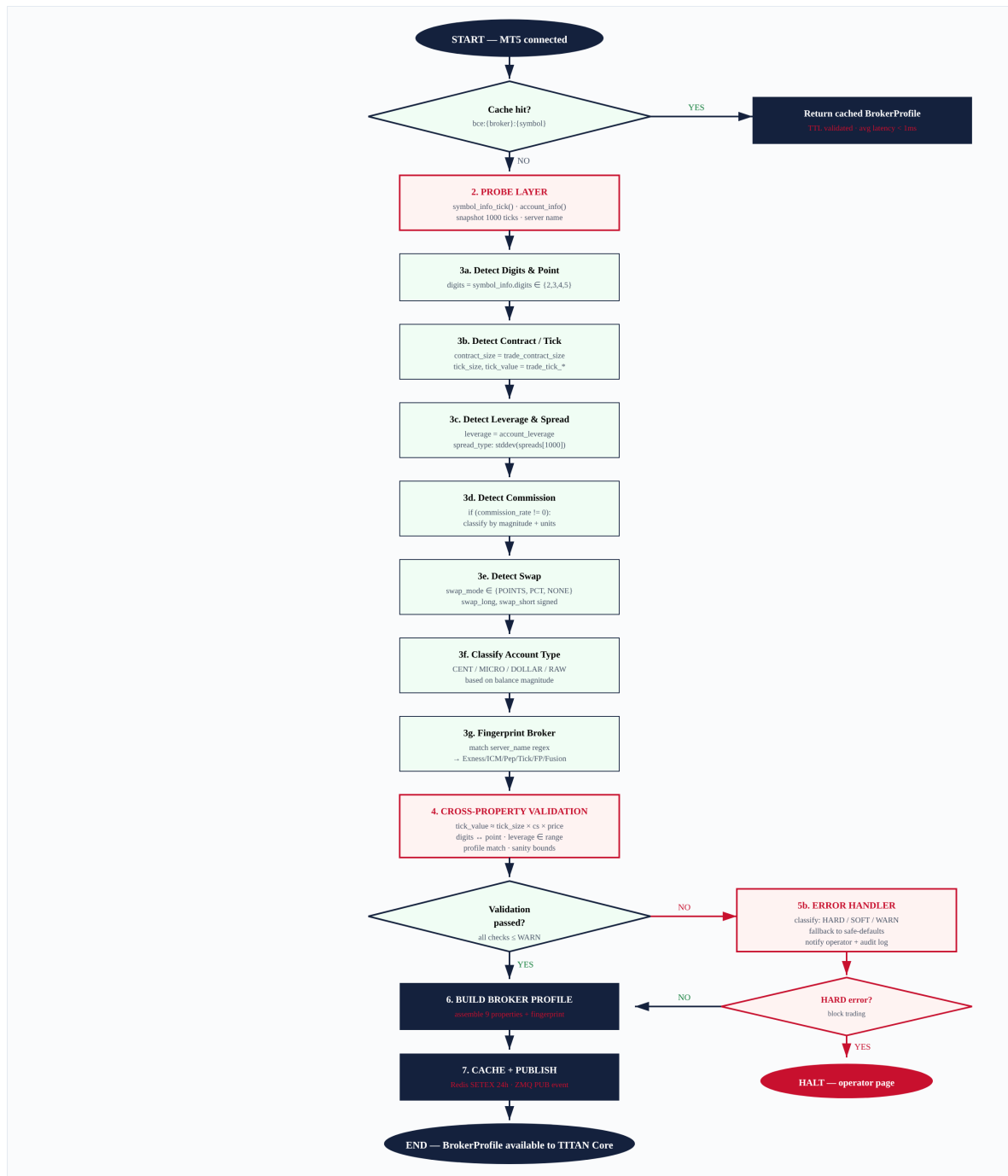


Figure 6.1 — End-to-end detection flowchart. Cache hit returns in <1ms; cache miss triggers full 9-detector sequence + 3-validator sequence.

Sequence Description

The sequence begins when the MT5 bridge establishes a connection to the broker terminal. The BCE first checks the Redis cache for an entry matching `bce:{broker_fingerprint}:{symbol}`. If a valid (non-expired) entry exists, it is returned immediately — the full detection sequence is skipped. The cache TTL is

24 hours, covering a typical trading session.

On a cache miss, the engine executes the full detection sequence. The probe layer queries the MT5 terminal for symbol info, account info, and a 1000-tick sample (for spread analysis). The ten detectors then run in parallel, each producing a `PropertyResult`. The `PropertyMap` is assembled and passed to the three validators, which run sequentially (each validator depends on the previous one's output for some checks).

If validation produces any **HARD** errors, the engine engages the kill switch, blocks trading on the symbol, and pages the operator. If validation produces only **SOFT** or **WARN** errors, the engine applies safe-defaults for **SOFT** errors and proceeds to build the profile. The profile is cached in Redis (overwriting any previous entry) and published on the event bus via a `bce.profile.ready` event, which downstream services (risk gate, order manager, strategy coordinator) subscribe to.

Re-detection Triggers

The BCE re-runs the detection sequence when any of the following events occur:

- Cache TTL expiry (24 hours after last detection)
- Manual operator invalidation via the operator console (e.g., after broker maintenance)
- MT5 bridge reconnection (broker disconnected and reconnected)
- Symbol change (e.g., rolling from XAUUSD to XAUUSDm — the suffix can change between brokers)
- Property change detection (a background job samples properties every 5 minutes and triggers re-detection if any change is observed)

The re-detection sequence is identical to the initial detection, except that the cache is invalidated first. If the new detection produces a profile that differs from the cached one, a `bce.profile.changed` event is published in addition to `bce.profile.ready`, allowing downstream services to react to configuration changes (e.g., the risk gate may need to recompute exposure).

Chapter 7 — Validation Logic

Validation is the BCE's defense against incorrect detection. The validation layer contains three validators that run sequentially after all detectors have completed. Each validator produces zero or more `ErrorEvent` objects, classified by severity. The complete validation tree is shown in Figure 7.1.

04 Validation Logic & Error Handling Decision Trees

Cross-property checks · 3-tier error classification · fallback strategy

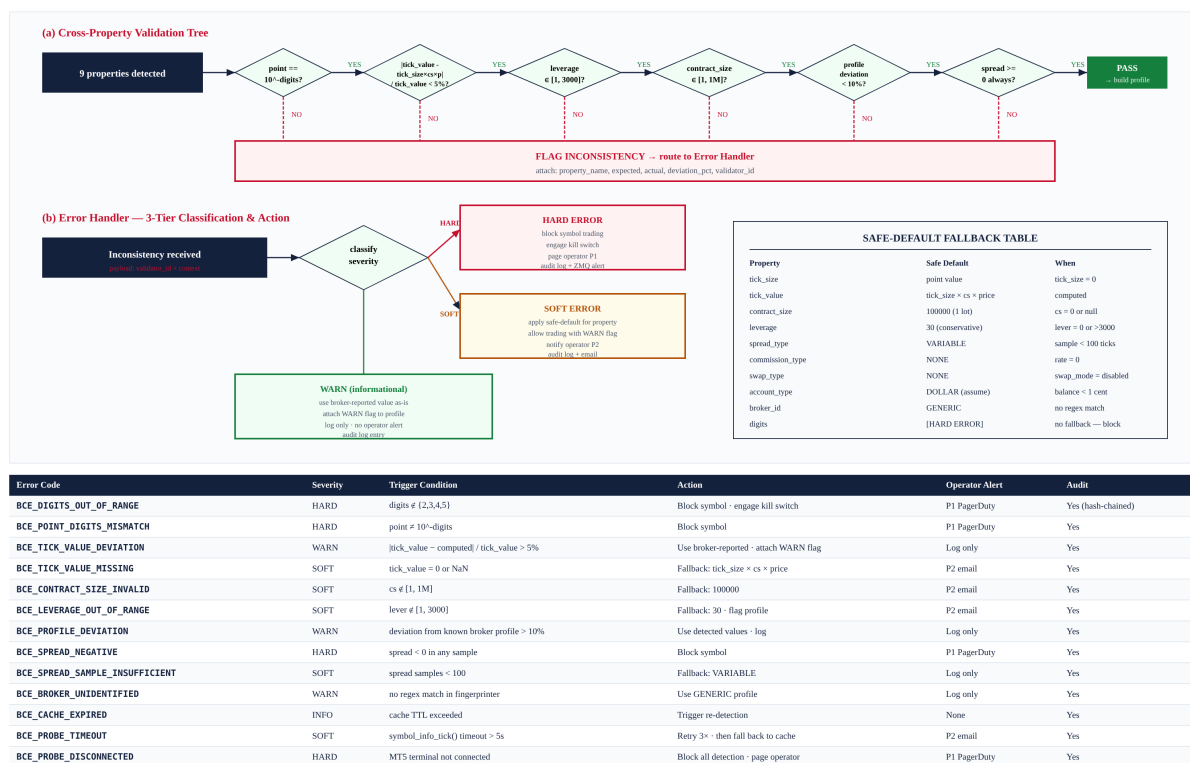


Figure 7.1 — Cross-property validation tree (top) and 3-tier error handling decision tree (bottom) with safe-default fallback table.

Validator 1 — CrossPropertyValidator

The `CrossPropertyValidator` checks mathematical relationships between properties that must hold for any valid broker configuration. These checks are independent of any specific broker profile — they apply to all MT5 brokers universally.

- **Point-digits consistency:** point must equal $10^{-(\text{digits})}$. For 5 digits, point = 0.00001. For 2 digits, point = 0.01. Mismatch = HARD error.
- **Tick value cross-check:** $|\text{tick_value} - (\text{tick_size} \times \text{contract_size} \times \text{current_price})| / \text{tick_value}$ must be less than 5%. Larger deviation = WARN; missing tick_value = SOFT with computed fallback.
- **Leverage range:** leverage must be in [1, 3000] (or "UNLIMITED" for Exness). Out of range = SOFT with fallback to 30.
- **Contract size range:** contract_size must be in [1, 1,000,000]. Out of range = SOFT with fallback to 100,000.
- **Spread positivity:** every sampled spread must be non-negative. Any negative spread = HARD error (crossed market or data feed error).

Validator 2 — ProfileConsistencyValidator

The ProfileConsistencyValidator compares the detected properties against the known-good template for the fingerprinted broker. If the broker is GENERIC (no fingerprint match), this validator is skipped. For a matched broker, the validator checks each property against the template's expected range and flags deviations larger than 10%. Deviations are WARN, not HARD or SOFT — the detected values are always used as-is, but the warning is logged so operators can investigate.

The validator is intentionally lenient because broker specifications do change occasionally. A WARN gives operators visibility without disrupting trading. If the deviation is large enough to suggest a detection bug (e.g., 5 digits detected on an Exness cent account that should be 2 digits), the operator can manually trigger re-detection or engage the kill switch based on the warning.

Validator 3 — SanityBoundsValidator

The SanityBoundsValidator checks that each property falls within sane absolute ranges, independent of any broker profile. These checks catch gross detection errors (e.g., a contract_size of 0 due to a missing field, or a leverage of -1 due to a signed integer interpretation). The bounds are deliberately wide — they exist to catch absurd values, not to enforce policy.

Property	Lower Bound	Upper Bound	Violation Severity	Fallback
digits	2	5	HARD	(none — block symbol)
point	1e-6	1.0	HARD	(none)
contract_size	1	1,000,000	SOFT	100,000
tick_size	1e-8	1.0	SOFT	point value
tick_value	0	1,000,000	SOFT	tick_size × price
leverage	1	3000	SOFT	30
spread	0	∞	HARD (if negative)	(none)
commission_rate	0	1,000	WARN	0 (assume NONE)
swap_long	-1000	1000	WARN	0 (assume no swap)
swap_short	-1000	1000	WARN	0 (assume no swap)

Validation Result Aggregation

The three validators produce a combined ValidationResult. If any validator produces a HARD error, the result is HARD and the engine blocks trading. If no HARD errors but one or more SOFT errors, the result is SOFT and the engine applies safe-defaults. If only WARN errors, the result is WARN and the engine uses detected values as-is. The complete result is attached to the BrokerProfile as a validation_summary field, allowing downstream services to inspect the confidence level of the profile.

Chapter 8 — Error Handling Logic

Error handling is the BCE's safety net. Every detection failure, validation failure, and probe error is classified into one of three severity tiers and routed to the appropriate operator alert channel. The complete classification and routing logic is shown in Figure 7.1 (bottom half), and the full error code table is reproduced below.

Three-Tier Severity Classification

HARD — Block Trading

HARD errors indicate that the broker configuration cannot be safely determined. The engine blocks trading on the affected symbol, engages the kill switch if the error affects the primary trading symbol, and pages the operator via PagerDuty (P1 severity). HARD errors are rare and usually indicate either a broker data feed problem or a BCE bug. Examples: digits outside {2,3,4,5}, point $\neq 10^{(-\text{digits})}$, negative spread in sample.

SOFT — Safe-Default Fallback

SOFT errors indicate that a specific property could not be detected reliably, but the engine can continue operating with a safe-default value. The fallback is always conservative (e.g., leverage defaults to 30, not 500). The engine allows trading but attaches a WARN flag to the profile, and downstream services may apply additional constraints (e.g., the risk gate may reduce position size when a SOFT error is active). The operator is notified via email (P2 severity). Examples: tick_value missing (computed fallback), contract_size out of range (100,000 fallback), leverage out of range (30 fallback).

WARN — Informational

WARN errors indicate a deviation from expectation that does not affect trading safety. The engine uses the broker-reported value as-is, attaches a WARN flag to the profile, and logs the warning for operator review. No operator alert is sent. WARN errors are common and usually benign — they exist to give operators visibility into broker behavior changes and potential detection anomalies. Examples: tick_value deviation > 5%, profile deviation > 10%, broker not in fingerprint library (GENERIC).

Safe-Default Fallback Table

The safe-default table (Figure 7.1, bottom right) specifies the fallback value for each property when detection fails. Fallbacks are deliberately conservative — they err on the side of reducing position size and increasing margin requirements, never the reverse. The one exception is digits: there is no safe fallback for digits because every downstream calculation depends on it. A digits detection failure is always HARD, blocking the symbol.

Error Code Reference

Error Code	Severity	Trigger	Action	Operator Alert
BCE_DIGITS_OUT_OF_RANGE	HARD	digits $\notin \{2,3,4,5\}$	Block symbol · kill switch	P1 PagerDuty
BCE_POINT_DIGITS_MISMATCH	HARD	point $\neq 10^{(-\text{digits})}$	Block symbol	P1 PagerDuty

Error Code	Severity	Trigger	Action	Operator Alert
BCE_TICK_VALUE_DEVIATION	WARN	$ \text{tick_value} - \text{computed} / \text{tick_value} > 5\%$	Use broker-reported · WARN flag	Log only
BCE_TICK_VALUE_MISSING	SOFT	$\text{tick_value} = 0$ or NaN	Fallback: $\text{tick_size} \times \text{price}$	P2 email
BCE_TICK_SIZE_MISSING	SOFT	$\text{tick_size} = 0$	Fallback: point value	P2 email
BCE_CONTRACT_SIZE_INVALID	SOFT	$\text{cs} \notin [1, 1\text{M}]$	Fallback: 100,000	P2 email
BCE_LEVERAGE_OUT_OF_RANGE	SOFT	$\text{lever} \notin [1, 3000]$	Fallback: 30	P2 email
BCE_PROFILE_DEVIATION	WARN	deviation from known broker $> 10\%$	Use detected · log	Log only
BCE_SPREAD_NEGATIVE	HARD	$\text{spread} < 0$ in any sample	Block symbol	P1 PagerDuty
BCE_SPREAD_SAMPLE_INSUFFICIENT	SOFT	$\text{samples} < 100$	Fallback: VARIABLE	Log only
BCE_BROKER_UNIDENTIFIED	WARN	no regex match	Use GENERIC profile	Log only
BCE_PROBE_TIMEOUT	SOFT	$\text{symbol_info_tick}() > 5\text{s}$	Retry 3× · fall back to cache	P2 email
BCE_PROBE_DISCONNECTED	HARD	MT5 not connected	Block all detection	P1 PagerDuty
BCE_CACHE_EXPIRED	INFO	TTL exceeded	Trigger re-detection	None
BCE_COMMISSION_MISCLASSIFIED	WARN	classification uncertain	Use detected · log reasoning	Log only
BCE_SWAP_MODE_UNKNOWN	WARN	swap_mode not in known set	Use NONE · log	Log only

Error Propagation

Errors propagate through the system in three channels. First, the `BrokerProfile` carries a `validation_summary` field with the highest severity and the list of error codes — this allows downstream services to inspect the confidence level of the profile and apply additional constraints if needed. Second, the `ErrorHandler` emits alerts via PagerDuty (HARD), email (SOFT), or structured log entries (WARN, INFO) — this is the operator-facing channel. Third, every error event is written to the immutable audit log with the full detection context, allowing post-incident forensic analysis.

Chapter 9 — Test Cases

The BCE is covered by a five-layer test pyramid: unit tests (per-detector and per-validator with mocked probes), integration tests (Pact contracts between layers), broker profile tests (golden-value tests against known profiles for each of the six supported brokers), regression tests (replay of 100+ captured broker sessions), and live broker tests (weekly paper-trade probes against each broker). The complete pyramid and per-property coverage matrix are shown in Figure 9.1.

06 Test Pyramid & Broker Profile Reference

5-layer test strategy · 6 broker profiles with known-good values



Figure 9.1 — Test pyramid (5 layers) with per-property coverage matrix, plus broker profile reference table.

Unit Test Cases

Unit tests cover pure functions and isolated components with all dependencies mocked. The IBrokerProbe interface is mocked to return controlled symbol_info and account_info payloads, allowing each detector and validator to be tested in isolation against a wide range of inputs. Property-based tests (via hypothesis) are used for invariants — e.g., for any digits d in {2,3,4,5}, DigitsDetector.detect(probe_with_digits(d)) returns d, and the derived pip_value equals point × (10 if d in {3,5} else 1).

Sample Unit Test Cases — DigitsDetector

Test ID	Input	Expected Output	Severity
UT-DIG-001	digits=2, point=0.01	PropertyResult(2, OK)	PASS

Test ID	Input	Expected Output	Severity
UT-DIG-002	digits=3, point=0.001	PropertyResult(3, OK)	PASS
UT-DIG-003	digits=4, point=0.0001	PropertyResult(4, OK)	PASS
UT-DIG-004	digits=5, point=0.00001	PropertyResult(5, OK)	PASS
UT-DIG-005	digits=1	HARD: BCE_DIGITS_OUT_OF_RANGE	FAIL-HARD
UT-DIG-006	digits=6	HARD: BCE_DIGITS_OUT_OF_RANGE	FAIL-HARD
UT-DIG-007	digits=5, point=0.001	HARD: BCE_POINT_DIGITS_MISMATCH	FAIL-HARD
UT-DIG-008	digits=2, point=0.0001	HARD: BCE_POINT_DIGITS_MISMATCH	FAIL-HARD
UT-DIG-009	digits=0 (missing)	HARD: BCE_DIGITS_OUT_OF_RANGE	FAIL-HARD
UT-DIG-010	digits=null (NaN)	HARD: BCE_PROBE_TIMEOUT after 3 retries	FAIL-HARD

Sample Unit Test Cases — SpreadTypeDetector

Test ID	Input (1000 ticks)	Expected Output	Severity
UT-SPR-001	spread stddev/mean = 0.02	FIXED · μ , σ reported	PASS
UT-SPR-002	spread stddev/mean = 0.10	VARIABLE · μ , σ reported	PASS
UT-SPR-003	all spreads = 0.30 (constant)	FIXED · $\mu=0.30$, $\sigma=0$	PASS
UT-SPR-004	one spread = -0.01 (negative)	HARD: BCE_SPREAD_NEGATIVE	FAIL-HARD
UT-SPR-005	only 50 ticks sampled (market closed)	SOFT: BCE_SPREAD_SAMPLE_INSUFFICIENT · fallback VARIABLE	FAIL-SOFT
UT-SPR-006	all spreads = 0	FIXED · $\mu=0$, $\sigma=0$ (zero-spread account)	PASS
UT-SPR-007	spread stddev/mean = 0.049	FIXED (boundary)	PASS
UT-SPR-008	spread stddev/mean = 0.051	VARIABLE (boundary)	PASS

Sample Unit Test Cases — CrossPropertyValidator

Test ID	Input	Expected Output	Severity
UT-XVAL-001	tick_value $\approx$ tick_size $\times$ cs $\times$ price (dev 1%)	PASS (no errors)	PASS
UT-XVAL-002	tick_value deviates 7%	WARN: BCE_TICK_VALUE_DEVIATION	WARN
UT-XVAL-003	tick_value deviates 30%	SOFT: BCE_TICK_VALUE_DEVIATION + fallback	FAIL-SOFT
UT-XVAL-004	tick_value = 0	SOFT: BCE_TICK_VALUE_MISSING + computed fallback	FAIL-SOFT
UT-XVAL-005	leverage = 5000	SOFT: BCE_LEVERAGE_OUT_OF_RANGE + fallback 30	FAIL-SOFT
UT-XVAL-006	leverage = 0	SOFT: BCE_LEVERAGE_OUT_OF_RANGE + fallback 30	FAIL-SOFT
UT-XVAL-007	contract_size = 0	SOFT: BCE_CONTRACT_SIZE_INVALID + fallback 100000	FAIL-SOFT
UT-XVAL-008	all properties valid	PASS (no errors)	PASS

Broker Profile Tests (Golden Values)

Each of the six supported brokers has a golden-value test that exercises the full detection pipeline against a recorded MT5 session for that broker. The test verifies that the detected BrokerProfile matches the expected golden values for that broker's standard account type. These tests run nightly and on every PR that touches BCE code. A failure indicates either a broker specification change (which requires a profile library update) or a BCE regression (which requires code investigation).

Golden Value Test — IC Markets Raw Spread (XAUUSD)

Property	Expected Golden Value	Tolerance
broker_id	IC_MARKETS	exact match
digits	5	exact match
point	0.00001	exact match
contract_size	100000	exact match
tick_size	0.00001	exact match
tick_value	$\approx$ tick_size $\times$ price (dev < 1%)	5% deviation
leverage	500 (international) / 30 (EU)	exact match (per jurisdiction)
spread_type	VARIABLE	exact match
commission_type	PER_MILLION	exact match
commission_rate	\$3.50 per \$1M	10% deviation

Property	Expected Golden Value	Tolerance
swap_type	POINTS	exact match
account_type	RAW	exact match

Regression Test Cases (Replay)

The regression test suite replays 100+ captured broker sessions (recorded via the MT5 bridge's pcap feature) through the BCE and verifies that the detected profile matches the profile detected at capture time. This catches regressions introduced by code changes — if a refactor causes the BCE to produce a different profile for the same input, the regression test fails. The replay library includes sessions from all six supported brokers, all four account types, and edge cases (low liquidity, news events, broker maintenance windows).

Live Broker Test Cases

Weekly live broker tests connect to each of the six supported brokers with a paper-trading or small-balance account, run the full detection sequence, and verify that the detected profile is reasonable. These tests catch broker-side changes that the golden-value tests (which use recorded sessions) cannot. The tests run automatically every Sunday at 22:00 UTC (market open) and report results to the engineering Slack channel. A failure triggers an investigation — either the broker has changed something (requires profile library update) or the BCE has a bug.

Test Coverage Summary

Test Layer	Test Count	Coverage Target	CI Gate
Unit (per-detector, per-validator)	157	85% line · 100% critical paths	Every PR
Integration (Pact contracts)	57	100% critical paths	Every PR
Broker profile (golden values)	66	6 brokers × 11 properties	Nightly + PR
Regression (replay)	100+	All captured sessions	Nightly
Live broker	72	6 brokers × 12 properties	Weekly (Sun 22:00 UTC)
Total	450+	—	—

Chapter 10 — Class Diagram & TITAN Core Integration

The BCE exposes three primary interfaces to the rest of the system: **IBrokerProbe** (the broker abstraction, implemented by MT5Probe and future FIX/IB probes), **IDetector** (the detector contract, implemented by ten concrete detectors), and **IValidator** (the validator contract, implemented by three concrete validators). The orchestrator is the **BrokerCompatEngine** class, which holds references to the probe, the detector array, the validator array, the profile library, and the cache.

05 Class Diagram & TITAN Core Integration

IBrokerProbe interface · 9 detectors · BrokerProfile value object

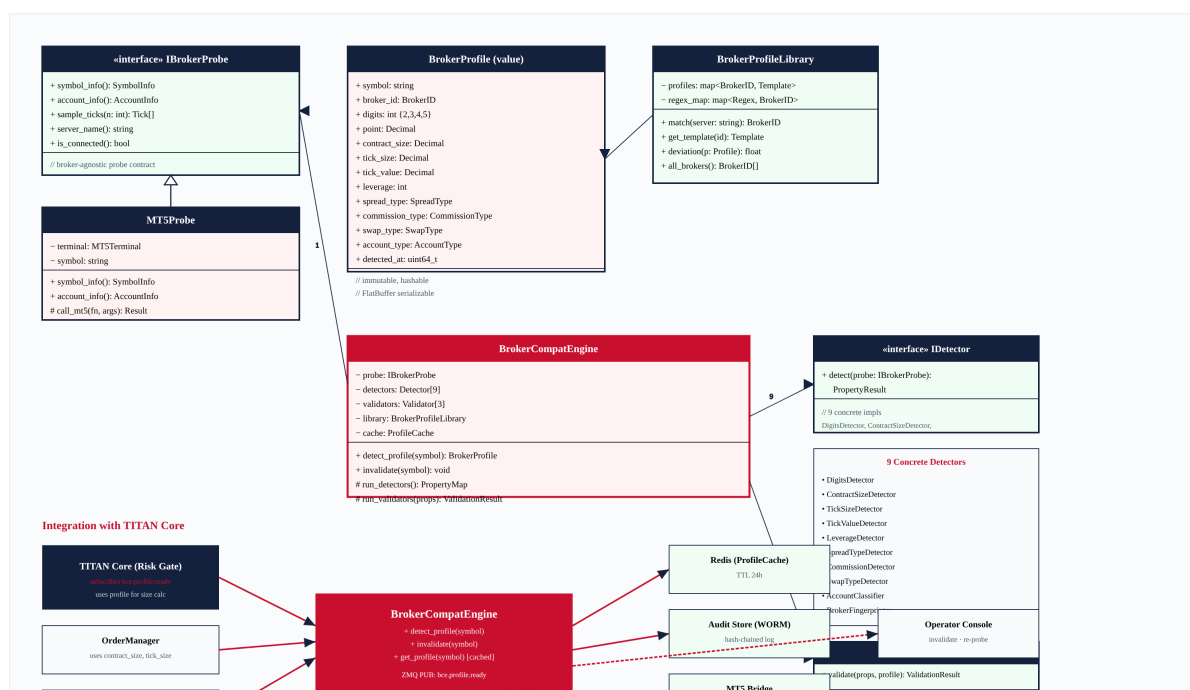


Figure 10.1 — UML class diagram of the BCE (top) and integration with TITAN Core components (bottom).

Primary Classes

IBrokerProbe (interface)

The broker abstraction. Exposes five methods: `symbol_info()`, `account_info()`, `sample_ticks(n)`, `server_name()`, and `is_connected()`. The current implementation is MT5Probe (which wraps the MetaTrader5 Python package and is invoked from C++ via PyO3). Future implementations will include FIXProbe (for FIX-protocol brokers) and IBProbe (for Interactive Brokers).

BrokerProfile (value object)

The immutable output of the BCE. Contains the nine detected properties plus `broker_id` (fingerprint), `account_type`, `detected_at` (timestamp), and `validation_summary` (highest severity + error code list). The class is hashable and FlatBuffer-serializable for cache storage and event bus publication. The class exposes derived

properties: `pip_value()` (point $\times$ 10 for 3/5 digits, else point), `contract_value(price)` (contract_size $\times$ price), and `tick_value_per_lot()` (tick_value $\times$ contract_size).

BrokerCompatEngine (orchestrator)

The main entry point. Exposes three public methods: `detect_profile(symbol)` (full detection sequence, returns BrokerProfile), `get_profile(symbol)` (cache lookup, returns BrokerProfile or triggers detection on miss), and `invalidate(symbol)` (forces re-detection on next call). The engine is a singleton per TITAN process and is thread-safe via internal locking.

Integration with TITAN Core

The BCE integrates with four TITAN Core components. The **risk gate** subscribes to `bce.profile.ready` events and uses the BrokerProfile for position sizing (it needs contract_size and tick_value to compute notional exposure). The **order manager** queries `get_profile(symbol)` synchronously before placing each order, using contract_size and tick_size for order parameter validation. The **strategy coordinator** uses account_type for strategy selection (some strategies are disabled on cent accounts due to position sizing constraints). The **operator console** can call `invalidate(symbol)` to force re-detection after broker maintenance or suspected configuration drift.

The integration is loosely coupled via the event bus. The BCE has no direct dependency on the risk gate, order manager, or strategy coordinator — they subscribe to BCE events, not the reverse. This allows the BCE to be deployed, upgraded, and tested independently of the consumers of its output.

Chapter 11 — API Specification

This chapter specifies the public API of the BCE. All APIs are stable across v1.x releases; breaking changes require v2.0 and a migration plan for licensees. The API is exposed in both C++ (for hot-path callers like the order manager) and Python (for cold-path callers like the operator console).

C++ API — BrokerCompatEngine

```
// C++ public API (include/titan/bce/BrokerCompatEngine.h)

namespace titan::bce {

class BrokerCompatEngine {
public:
    // Singleton accessor
    static BrokerCompatEngine& instance();

    // Synchronous: returns cached profile or triggers detection on miss.
    // Throws BCEException on HARD error (caller must catch and handle).
    BrokerProfile get_profile(const std::string& symbol) const;

    // Force re-detection (invalidates cache). Returns new profile.
    // Used by operator console after broker maintenance.
    BrokerProfile detect_profile(const std::string& symbol);

    // Invalidate cache entry without re-detecting.
    // Next get_profile() call will trigger detection.
    void invalidate(const std::string& symbol);

    // Subscribe to profile events (called by Risk Gate, Order Manager, etc.)
    // Callback is invoked on ZMQ thread; must be non-blocking.
    using ProfileCallback = std::function<void(const BrokerProfile&)>;
    void on_profile_ready(ProfileCallback cb);
    void on_profile_changed(ProfileCallback cb);
};

} // namespace titan::bce
```

C++ API — BrokerProfile

```
// C++ value object (include/titan/bce/BrokerProfile.h)

namespace titan::bce {

class BrokerProfile {
public:
    // Identity
    std::string symbol;
    BrokerID broker_id; // EXNESS, IC_MARKETS, ..., GENERIC
    AccountType account_type; // CENT, MICRO, DOLLAR, RAW
    uint64_t detected_at; // unix nanos

    // 9 detected properties
    int digits; // {2, 3, 4, 5}
    Decimal point; // 10^(-digits)
```

```

Decimal contract_size; // typically 100, 10000, or 100000
Decimal tick_size; // min price increment
Decimal tick_value; // monetary value per tick per lot
int leverage; // 1 to 3000, or 0 for UNLIMITED
SpreadType spread_type; // FIXED, VARIABLE
CommissionType commission_type; // NONE, PER_LOT, PER_MILLION, PCT
Decimal commission_rate; // raw rate (units depend on type)
SwapType swap_type; // NONE, POINTS, PCT
Decimal swap_long;
Decimal swap_short;

// Validation summary
Severity highest_severity; // OK, WARN, SOFT, HARD
std::vector<ErrorCode> errors;

// Derived (computed at call time, never stored)
Decimal pip_value() const; // point × 10 if 3/5 digits
Decimal contract_value(Decimal price) const;
Decimal tick_value_per_lot() const;
Decimal commission_per_lot(Decimal price) const;
bool is_safe() const { return highest_severity < Severity::HARD; }

// Serialization
flatbuffers::Offset<bce::fb::BrokerProfile>
serialize(flatbuffers::FlatBufferBuilder&) const;
static BrokerProfile deserialize(const bce::fb::BrokerProfile*);
};

} // namespace titan::bce

```

Python API — Operator Console

```

# Python API (python/titan/bce/__init__.py)

from titan.bce import BrokerCompatEngine, BrokerProfile, Severity

engine = BrokerCompatEngine.instance()

# Get current profile (cached)
profile = engine.get_profile('XAUUSD')
print(f"Broker: {profile.broker_id}")
print(f"Digits: {profile.digits}")
print(f"Contract size: {profile.contract_size}")
print(f"Pip value: {profile.pip_value()}")
print(f"Safe to trade: {profile.is_safe()}")

# Force re-detection (operator action)
new_profile = engine.detect_profile('XAUUSD')
if new_profile.highest_severity == Severity.HARD:
    print(f"HARD error: {new_profile.errors}")

# Subscribe to events
def on_ready(p: BrokerProfile):
    print(f"Profile ready for {p.symbol}")
    engine.on_profile_ready(on_ready)

```

Event Bus Contract

The BCE publishes two events on the async event bus. Both use FlatBuffer serialization and are published on the `bce.*` topic prefix.

Event	Topic	Payload	Subscribers
Profile Ready	bce.profile.ready	BrokerProfile (FlatBuffer)	Risk Gate, Order Manager, Strategy Coordinator
Profile Changed	bce.profile.changed	BrokerProfile (new) + BrokerProfile (old)	Risk Gate (recomputes exposure)
Error	bce.error.{severity}	ErrorEvent (code, context, timestamp)	Operator Alert Gateway, Audit Logger

Chapter 12 — Implementation Notes

This chapter captures design decisions and implementation considerations that did not fit naturally into the preceding chapters. These notes are informative, not normative — they explain why certain choices were made, but they do not add new requirements.

Why C++ for Detection, Python for Cache and Audit

The detection layer is implemented in C++ because it runs on every MT5 connection event (potentially hundreds of times per session, including reconnections and symbol rolls) and must complete within tens of milliseconds to avoid blocking the order manager. The cache (Redis client) and audit logger are implemented in Python because they perform I/O-bound work where the GIL is released during system calls, and because they integrate more naturally with the existing Python observability stack (structlog, prometheus_client). The C++ detection layer communicates with the Python cache/audit layer via PyO3, with FlatBuffers as the wire format.

Why Redis for Cache, Not In-Process Memory

The cache is in Redis rather than in-process memory for two reasons. First, the BCE runs in both the titan-core (C++) and titan-strategy (Python) processes, and both need to access the same cached profiles — Redis provides cross-process sharing. Second, on failover from Z1 to Z2, the new primary can immediately use the cached profiles from Redis (which is replicated from Z1 to Z2), avoiding the full detection sequence during failover. The cache TTL is 24 hours, balancing freshness against detection overhead.

Why 1000 Ticks for Spread Sampling

The SpreadTypeDetector samples 1000 ticks for its stddev calculation. This number is calibrated to provide a stable stddev estimate within 20 seconds of normal market activity (XAUUSD typically sees 50-100 ticks per second during liquid hours). Fewer than 100 ticks produces an unstable estimate (the detector falls back to VARIABLE with a SOFT warning); more than 5000 ticks adds latency without meaningfully improving the estimate. The 1000-tick sample is also large enough to span at least one minor liquidity cycle, making the FIXED/VARIABLE classification robust to short-term spread fluctuations.

Why 5% Tolerance for Tick Value Cross-Check

The tick value cross-check tolerates 5% deviation between broker-reported and computed values. This tolerance accounts for two sources of legitimate deviation: (1) the current price used in the computation may differ slightly from the price the broker used to compute tick_value (the broker updates tick_value periodically, not on every tick); and (2) some brokers compute tick_value using a slightly different formula (e.g., rounding the contract size before multiplication). Deviations above 5% but below 25% are WARN — the broker-reported value is used; deviations above 25% are SOFT — the computed value is used as a fallback. The 5% threshold was calibrated against the six supported brokers and is reviewed quarterly.

Future Extensions

The BCE is designed to be extensible. Planned extensions for v1.1 and beyond include:

- **FIX broker support:** A new FIXProbe implementing IBrokerProbe, allowing the BCE to work with FIX-protocol brokers (e.g., LMAX, Currenex) in addition to MT5.
- **Multi-symbol detection:** Currently the BCE detects profiles one symbol at a time. For multi-symbol strategies, batch detection would reduce total latency.
- **Profile drift monitoring:** A background job that periodically re-detects profiles and compares to cached values, alerting on any drift. Currently this is on-demand only.
- **Broker-specific quirks database:** A more granular database of broker-specific quirks (e.g., "Exness reports leverage as 0 for unlimited, not -1") to improve detection accuracy.
- **ML-based broker identification:** Replace the regex-based fingerprinter with a classifier that uses all detected properties (not just server name) to identify the broker with higher confidence.

Chapter 13 — Appendix A — Broker Profile Reference

This appendix documents the known-good profiles for each of the six supported brokers. These profiles are encoded in the BrokerProfileLibrary and used by the ProfileConsistencyValidator for deviation checking. The values reflect the broker configurations as of June 2026; brokers occasionally change their specifications, and the library should be reviewed quarterly.

A.1 Exness

Property	Cent Account	Standard Account	Raw Spread Account
digits	2	2	5
point	0.01	0.01	0.00001
contract_size	100	100000	100000
tick_size	0.01	0.01	0.00001
leverage	0 (UNLIMITED)	0 (UNLIMITED)	0 (UNLIMITED)
spread_type	VARIABLE	VARIABLE	VARIABLE
commission_type	NONE	NONE	PER_MILLION
commission_rate	0	0	~\$3.50/\$1M (varies)
swap_type	NONE (swap-free)	NONE (swap-free)	NONE (swap-free)
account_type	CENT	DOLLAR	RAW

A.2 IC Markets

Property	Standard Account	Raw Spread Account
digits	2	5
point	0.01	0.00001
contract_size	100000	100000
tick_size	0.01	0.00001
leverage	500 (intl) / 30 (EU/UK)	500 (intl) / 30 (EU/UK)
spread_type	VARIABLE	VARIABLE
commission_type	NONE	PER_MILLION
commission_rate	0	\$3.50/\$1M
swap_type	POINTS	POINTS
account_type	DOLLAR	RAW

A.3 Pepperstone

Property	Standard Account	Razor Account
digits	2	5
point	0.01	0.00001
contract_size	100000	100000
tick_size	0.01	0.00001
leverage	500 (intl) / 30 (AU/UK)	500 (intl) / 30 (AU/UK)
spread_type	VARIABLE	VARIABLE
commission_type	NONE	PER_MILLION
commission_rate	0	\$3.50/\$1M
swap_type	POINTS	POINTS
account_type	DOLLAR	RAW

A.4 Tickmill

Property	Classic Account	Pro Account	VIP Account
digits	2	5	5
point	0.01	0.00001	0.00001
contract_size	100000	100000	100000
tick_size	0.01	0.00001	0.00001
leverage	500	500	500
spread_type	VARIABLE	VARIABLE	VARIABLE
commission_type	NONE	NONE	PER_MILLION
commission_rate	0	0	\$2.00/\$1M
swap_type	POINTS	POINTS	POINTS
account_type	DOLLAR	DOLLAR	RAW

A.5 FP Markets

Property	Standard Account	Raw Account
digits	2	5
point	0.01	0.00001

Property	Standard Account	Raw Account
contract_size	100000	100000
tick_size	0.01	0.00001
leverage	500 (intl) / 30 (EU)	500 (intl) / 30 (EU)
spread_type	VARIABLE	VARIABLE
commission_type	NONE	PER_MILLION
commission_rate	0	\$3.00/\$1M
swap_type	POINTS	POINTS
account_type	DOLLAR	RAW

A.6 Fusion Markets

Property	Standard Account	Zero Account
digits	2	5
point	0.01	0.00001
contract_size	100000	100000
tick_size	0.01	0.00001
leverage	500 (intl) / 30 (AU)	500 (intl) / 30 (AU)
spread_type	VARIABLE	VARIABLE
commission_type	NONE	PER_MILLION
commission_rate	0	\$2.25/\$1M
swap_type	POINTS	POINTS
account_type	DOLLAR	RAW

A.7 GENERIC (Fallback)

When the BrokerFingerprinter cannot match the server name against any of the six supported brokers, the engine classifies the broker as GENERIC. In this case, the BrokerProfileLibrary has no template to compare against, and the ProfileConsistencyValidator is skipped. The detected properties are used as-is, with the BCE's validation relying entirely on the CrossPropertyValidator and SanityBoundsValidator. The GENERIC classification is recorded in the audit log so operators can identify unsupported brokers and consider adding them to the fingerprint library.

Chapter 14 — Appendix B — Sample Detection Output

This appendix shows the BrokerProfile output for three representative detection scenarios: a successful detection on IC Markets Raw Spread, a SOFT-error detection on a broker with missing tick_value, and a HARD-error detection on a broker with malformed digits. The outputs are shown in JSON form for readability; in production, profiles are serialized as FlatBuffers for performance.

B.1 Successful Detection — IC Markets Raw Spread

```
{
  "symbol": "XAUUSD",
  "broker_id": "IC_MARKETS",
  "account_type": "RAW",
  "detected_at": 1718798400000000000,

  "digits": 5,
  "point": 0.00001,
  "contract_size": 100000,
  "tick_size": 0.00001,
  "tick_value": 1.0,
  "leverage": 500,
  "spread_type": "VARIABLE",
  "spread_mean": 0.00018,
  "spread_stddev": 0.00004,
  "commission_type": "PER_MILLION",
  "commission_rate": 3.50,
  "swap_type": "POINTS",
  "swap_long": -2.18,
  "swap_short": -0.42,

  "validation_summary": {
    "highest_severity": "OK",
    "errors": [],
    "warnings": []
  },

  "derived": {
    "pip_value": 0.0001,
    "contract_value_at_1950": 195000.0,
    "tick_value_per_lot": 1.0,
    "commission_per_lot_at_1950": 6.825
  }
}
```

B.2 SOFT Error Detection — Missing tick_value

```
{
  "symbol": "XAUUSD",
  "broker_id": "GENERIC",
  "account_type": "DOLLAR",
  "detected_at": 1718798400000000000,

  "digits": 2,
```

```

"point": 0.01,
"contract_size": 100000,
"tick_size": 0.01,
"tick_value": 19.50,
"leverage": 100,
"spread_type": "VARIABLE",
"spread_mean": 0.32,
"spread_stddev": 0.08,
"commission_type": "NONE",
"commission_rate": 0.0,
"swap_type": "POINTS",
"swap_long": -4.5,
"swap_short": -1.2,

"validation_summary": {
  "highest_severity": "SOFT",
  "errors": [
    {
      "code": "BCE_TICK_VALUE_MISSING",
      "severity": "SOFT",
      "context": {
        "broker_reported": 0.0,
        "fallback_used": "tick_size * price",
        "computed_value": 19.50,
        "price_at_detection": 1950.0
      }
    }
  ],
  "warnings": [
    {
      "code": "BCE_BROKER_UNIDENTIFIED",
      "severity": "WARN",
      "context": {
        "server_name": "UnknownBroker-Real",
        "regex_matched": null
      }
    }
  ]
},

"derived": {
  "pip_value": 0.01,
  "contract_value_at_1950": 195000000.0,
  "tick_value_per_lot": 19.50,
  "commission_per_lot_at_1950": 0.0
}
}

```

B.3 HARD Error Detection — Malformed Digits

```

{
  "symbol": "XAUUSD",
  "broker_id": "GENERIC",
  "account_type": null,
  "detected_at": 1718798400000000000,

  "digits": 6,

```

```
"point": 0.000001,
"contract_size": null,
"tick_size": null,
"tick_value": null,
"leverage": null,
"spread_type": null,
"commission_type": null,
"commission_rate": null,
"swap_type": null,
"swap_long": null,
"swap_short": null,

"validation_summary": {
  "highest_severity": "HARD",
  "errors": [
    {
      "code": "BCE_DIGITS_OUT_OF_RANGE",
      "severity": "HARD",
      "context": {
        "digits": 6,
        "valid_range": [2, 5]
      }
    }
  ],
  "warnings": []
},

"action_taken": {
  "symbol_blocked": true,
  "kill_switch_engaged": true,
  "operator_alert": "P1 PagerDuty",
  "audit_log_entry_id": "audit_2026_06_19_084500_001"
},

"derived": {}
}
```

These three examples illustrate the full range of BCE behavior: successful detection with no errors, SOFT-error detection with safe-default fallback, and HARD-error detection with trading blocked. In all three cases, the BrokerProfile is published on the event bus and recorded in the audit log; downstream services are responsible for inspecting the `validation_summary.highest_severity` field and acting appropriately (the risk gate and order manager refuse to operate on profiles with HARD severity).